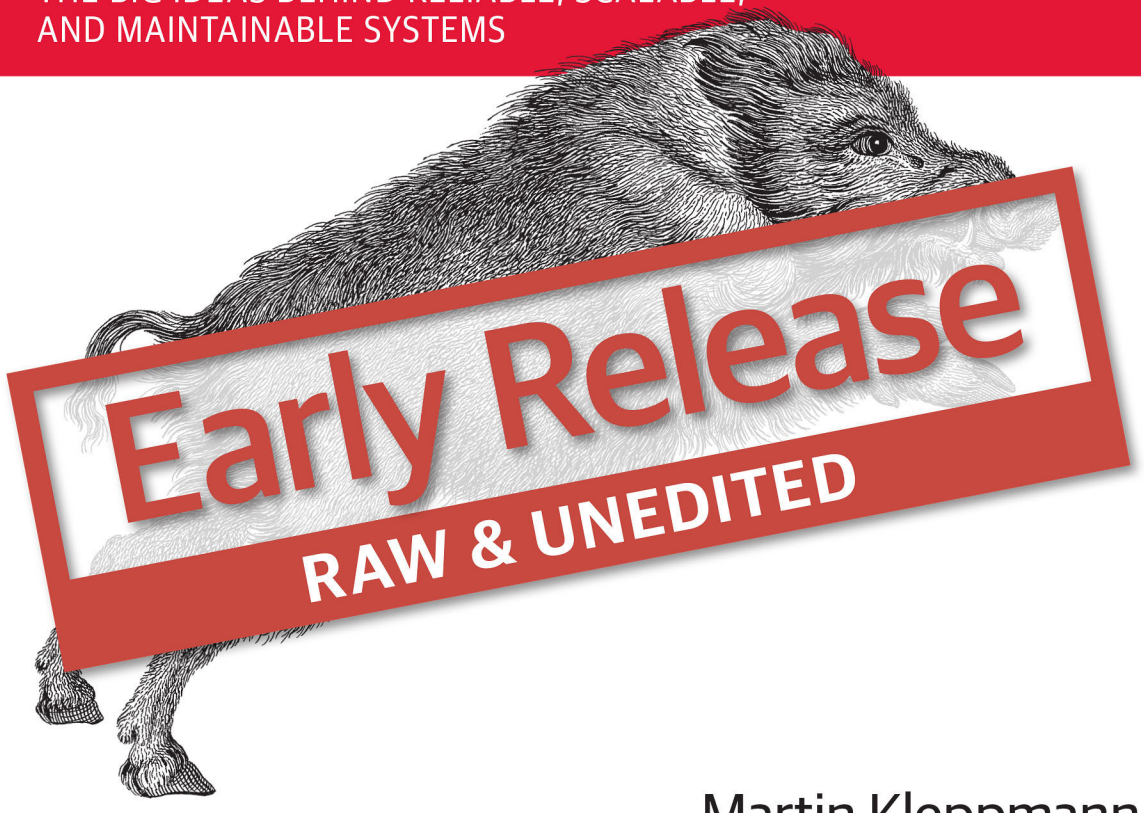
O'REILLY®

Designing Data-Intensive Applications

THE BIG IDEAS BEHIND RELIABLE, SCALABLE, AND MAINTAINABLE SYSTEMS



Martin Kleppmann

Designing Data-Intensive Applications
by Martin Kleppmann

Printed in the United States of America.

Published by O'Reilly Media, Inc. , 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (*http://safaribooksonline.com*). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com .

Editors: Mike Loukides and Ann Spencer
Production Editor: FILL IN PRODUCTION EDITOR

Copyeditor: FILL IN COPYEDITOR
Proofreader: FILL IN PROOFREADER
Indexer: FILL IN INDEXER
Interior Designer: David Futato
Cover Designer: Karen Montgomery
Illustrator: Rebecca Demarest

January -4712: First Edition

Revision History for the First Edition
2015-03-06: First Early Release
2015-04-01: Second Early Release
2015-06-18: Third Early Release
2016-01-26: Fourth Early Release
2016-05-05: Fifth Early Release
2016-07-11: Sixth Early Release

See *http://oreilly.com/catalog/errata.csp?isbn=9781491903094* for release details.

978-1-491-90309-4

[FILL IN]

Designing Data-Intensive Applications

Martin Kleppmann

Beijing • Boston • Farnham • Sebastopol • Tokyo O'REILLY®

Table of Contents

Part II. Distributed Data

Technology is a powerful force in our society. Data, software and communication can be used for bad: to entrench unfair power structures, to undermine human rights, and to protect vested interests. But they can also be used for good: to make underrepresented people's voices heard, to create opportunities for everyone, and to avert disasters. This book is dedicated to everyone working towards the good.

> *Computing is pop culture. […] Pop culture holds a disdain for history. Pop culture is all about identity and feeling like you're participating. It has nothing to do with cooperation, the past or the future — it's living in the present. I think the same is true of most people who write code for money. They have no idea where [their culture came from]…*
>
> —*Alan Kay, Dr Dobb's Journal (2012)*

About this Book

If you have worked in software engineering in recent years, especially in server-side and backend systems, you have probably been bombarded with a plethora of buzzwords relating to storage and processing of data. NoSQL! Big Data! Web-scale! Sharding! Eventual consistency! ACID! CAP theorem! Cloud services! MapReduce! Real-time!

In the last decade we have seen many interesting developments in databases, distributed systems and in the ways we build applications on top of them. There are various driving forces for these developments, including:

- Internet companies such as Google, Yahoo!, Amazon, Facebook, LinkedIn and Twitter are handling huge volumes of data and traffic, forcing them to create new tools that enable them to efficiently handle such scale.
- Businesses need to be agile, test hypotheses cheaply, and respond quickly to new market insights, by keeping development cycles short and data models flexible.
- Free and open source software has become very successful, and is now preferred to commercial or bespoke in-house software in many environments.
- CPU clock speeds are barely increasing, but multi-core processors are standard, and networks are getting faster. This means parallelism is only going to increase.
- Even if you work on a small team, you can now build systems that are distributed across many machines and even multiple geographic regions, thanks to infrastructure as a service (IaaS) such as Amazon Web Services.
- Many services are now expected to be highly available; extended downtime due to outages or maintenance is becoming increasingly unacceptable.

Data-intensive applications are pushing the boundaries of what is possible by making use of these technological developments. We call an application *data-intensive* if data is its primary challenge: the quantity of data, the complexity of data, or the speed at

which it is changing (as opposed to compute-intensive, where CPU cycles are the bottleneck).

The tools and technologies that help data-intensive applications store and process data have been rapidly adapting to these changes. New types of database systems ("NoSQL") have been getting lots of attention, but message queues, caches, search indexes, frameworks for batch and stream processing, and related technologies are very important too. Many applications use some combination of these.

The buzzwords that fill this space are a sign of enthusiasm for the new possibilities, which is a great thing. However, as software engineers and architects, we also need to have a technically accurate and precise understanding of the various technologies and their trade-offs if we want to build good applications. For that understanding, we have to dig deeper than buzzwords.

Fortunately, behind the rapid changes in technology, there are enduring principles that remain true, no matter which version of a particular tool you are using. If you understand those principles, you're in a position to see where each tool fits in, how to make good use of it, and how to avoid its pitfalls. That's where this book comes in.

The goal of this book is to help you navigate the diverse and fast-changing landscape of technologies for processing and storing data. This book is not a tutorial for one particular tool, nor is it a textbook full of dry theory. Instead, we will look at examples of successful data systems: technologies that form the foundation of many popular applications, and that have to meet scalability, performance and reliability requirements in production every day.

We will dig into the internals of those systems, tease apart their key algorithms, discuss their principles and the trade-offs they have to make. On this journey, we will try to find useful ways of *thinking about* data systems — not just *how* they work, but also *why* they work that way, and what questions we need to ask.

After reading this book, you will be in a great position to decide which kind of technology is appropriate for which purpose, and understand how tools can be combined to form the foundation of a good application architecture. You won't be ready to build your own database storage engine from scratch, but fortunately that is rarely necessary. You will, however, develop a good intuition for what your systems are doing under the hood, so that you can reason about their behavior, make good design decisions, and track down any problems that may arise.

Who Should Read this Book?

If you develop applications that have some kind of server/backend for storing or processing data, and your applications use the internet (e.g. web applications, mobile apps, or internet-connected sensors), then this book is for you.

This book is for software engineers, software architects and technical managers who love to code. It is especially relevant if you need to make decisions about the architecture of the systems you work on — for example, if you need to choose tools for solving a given problem, and figure out how best to apply them. But even if you have no choice over your tools, this book will help you better understand their strengths and weaknesses.

You should have some experience building web-based applications or network services, and you should be familiar with relational databases and SQL. Any non-relational databases and other data-related tools you know are a bonus, but not required. A general understanding of common network protocols like TCP and HTTP is helpful. Your choice of programming language or framework makes no difference for this book.

If any of the following are true for you, you'll find this book valuable:

- You want to learn how to make data systems scalable, for example to support web or mobile apps with millions of users.
- You need to make applications highly available (minimizing downtime) and operationally robust.
- You are looking for ways of making systems easier to maintain in the long run, even as they grow, and as requirements and technologies change.
- You have a natural curiosity for the way things work, and want to know what goes on inside major websites and online services. This book breaks down the internals of various databases and data processing systems, and it's great fun to explore the bright thinking that went into their design.

Sometimes, when discussing scalable data systems, people make comments along the lines of *"you're not Google or Amazon, stop worrying about scale and just use a relational database"*. There is truth in that statement: building for scale that you don't need is wasted effort, and may lock you into an inflexible design. In effect, it is a form of premature optimization. However, it's also important to choose the right tool for the job, and different technologies each have their own strengths and weaknesses. As we shall see, relational databases are important, but not the final word on dealing with data.

Scope of this Book

This book does not attempt to give detailed instructions on how to install or use specific software packages or APIs, since there is already plenty of documentation for those things. Instead we discuss the various principles and trade-offs that are fundamental to data systems, and we explore the different design decisions taken by different products.

Most of what we discuss in this book has already been said elsewhere in some form or another — in conference presentations, research papers, blog posts, code, bug trackers, and engineering folklore. This book summarizes the most important ideas from many different sources, and it includes pointers to the original literature throughout the text. The references at the end of each chapter are a great resource if you want to explore an area in more depth.

We look primarily at the *architecture* of data systems and the ways how they are integrated into data-intensive applications. This book doesn't have space to cover deployment, operations, security, ethics, management and other areas — those are complex and important topics, and we wouldn't do them justice by making them superficial side-notes in this book. They deserve books of their own.

Many of the technologies described in this book fall within the realm of the *Big Data* buzzword. However, the term *Big Data* is so over-used and under-defined that it is not useful in a serious engineering discussion. This book uses less ambiguous terms, such as single-node vs. distributed systems, or online/interactive vs. offline/batch processing systems.

This book has a bias towards free and open source software (FOSS), because reading, modifying and executing source code is a great way to understand how something works in detail. Open platforms also reduce the risk of vendor lock-in. However, where appropriate, we also discuss proprietary software (closed-source software, software as a service, or companies' in-house software that is only described in literature but not released publicly).

Outline of this Book

This book is arranged into three parts:

1. In Part I, we will discuss the fundamental ideas that we need in order to design data-intensive applications. We'll start in Chapter 1 by discussing what we're actually trying to achieve: reliability, scalability and maintainability — how we need to think about them, and how we can achieve them. In Chapter 2 we will compare several different data models and query languages, and see how they are appropriate to different situations. In Chapter 3 we will talk about storage engines: how databases arrange data on disk so that you can find it again efficiently. Chapter 4 turns to formats for data encoding (serialization) and evolution of schemas over time.
2. In Part II, we will move from data stored on one machine to data that is distributed across multiple machines. This is often necessary for scalability, but brings with it a variety of unique challenges. We'll first discuss replication (Chapter 5), partitioning/sharding (Chapter 6), and transactions (Chapter 7). We will then go into more detail on the problems with distributed systems (Chapter 8)

and what it means to achieve consistency and consensus in a distributed system (Chapter 9).

3. In Part III, we move up another step and discuss building heterogeneous systems that consist of several different components. As there is no one database for all use cases, applications often need to integrate several different databases, caches, indexes and so on. In Chapter 10 we will start with a batch processing approach, and build upon this in Chapter 11 to describe stream processing. Finally, in ??? we will put everything together and discuss how we can integrate different data systems into reliable, scalable and maintainable applications.

Early Release Status and Feedback

This is an early release copy of *Designing Data-Intensive Applications*. The text, figures and examples are a work in progress, and several chapters are yet to be written. We are releasing the book before it is finished because we hope that it is already useful in its current form, and because we would love your feedback in order to create the best possible finished product.

If you find any errors or glaring omissions, if you find anything confusing, or if you have any ideas for improving the book, please email the author and editors at *feedback@dataintensive.net*.

PART I

Foundations of Data Systems

The first four chapters go through the fundamental ideas that apply to all data systems, whether running on a single machine or distributed across a cluster of machines:

1. Chapter 1 introduces the terminology and approach that we're going to use throughout this book. It examines what we actually mean with words like *reliability*, *scalability* and *maintainability*, and how we can try to achieve them.
2. Chapter 2 compares several different data models and query languages — the most visible difference between different databases from a developer's point of view. We will see how different models are appropriate to different situations.
3. Chapter 3 turns to the internals of storage engines, and looks at how databases lay out data on disk. Different storage engines are optimized for different workloads, and choosing the right one can have a huge effect on performance.
4. Chapter 4 compares various formats for data encoding (serialization), and especially examines how they fare in an environment where application requirements change and schemas need to adapt over time.

Later, Part II will turn to the particular issues of distributed data systems.